THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE
CS-4031 Compiler Construction — Fall 2026

Lecture 7
Transition Diagrams and Lexer Implementation Strategies: Ad Hoc vs. Table-Driven Scanning

CLO(s) covered:	CLO 1, CLO 3
Dragon Book [DB]:	§3.4 (Recognition of Tokens: transition diagrams for `relop`, identifiers/keywords, and whitespace; sketching a lexical analyzer directly from transition diagrams)
Other references:	[ET] Engineering a Compiler, §2.2–2.3 (hand-written scanners); [AP] Modern Compiler Implementation, §2.5

Lecture Roadmap

1. **Transition diagrams**: a lighter-weight, one-pattern-at-a-time cousin of the DFAs built in Lecture **??** – exactly the tool DB §3.4 uses to go from "here is a regular expression" to "here is code," worked through on `relop`, identifiers/keywords, and whitespace.
2. **Retraction**: what it means for a state to need to "look one character too far ahead," and how that connects directly back to Lecture **??**'s double-buffering scheme.
3. **The ad hoc approach**: translating a set of transition diagrams straight into hand-written scanning code, one routine per token category, the way many real compilers' lexers are actually built.
4. **Ad hoc vs. table-driven, head to head**: the two genuinely different ways to *implement* a lexer – hand-coded diagrams (today) vs. the generic, generated DFA simulation Lecture **??** already built in full – and when a real engineer reaches for each one.

Lectures **??**–**??** gave you the *formal* machinery (NFA, subset construction, minimization) that a lexer-generator tool runs automatically. This lecture asks a different, very practical question: what if you are writing the scanner *by hand*, without a generator at all? DB introduces transition diagrams for exactly this reason, and – not coincidentally – this is also where Project Phase 1 and Assignment 1 begin.

7.1 From Formal Automata to a Concrete Scanner

[DB] DB actually introduces transition diagrams (§3.4) *before* it formalizes NFAs and DFAs – historically, hand-drawn diagrams like these are how lexers were built for years before subset construction and minimization were automated into generator tools. Having already done Lectures **??**–**??** in full formal generality, we are in a good position to see transition diagrams for what they really are: **a deliberately simplified, by-hand DFA**, drawn separately for *one* token category at a time, skipping the machinery (§**??**'s tagged-union combination, full subset construction) that a generator needs to merge *many* categories into a single automaton automatically.

Why Bother, Given Lecture ?? Already Solved This?

Lecture **??** answered "how does a lexer-generator tool build a scanner automatically." This lecture answers a different, equally real question: "what does a programmer actually *write*, by hand, when no generator tool is used at all?" Both questions matter – production compilers are

built both ways, sometimes even the *same* compiler mixing both strategies for different parts of its lexer (§7.6 returns to this). Understanding the hand-written route also makes the generated route feel less like magic: every diagram in this lecture is exactly the kind of small DFA fragment that subset construction (§??) would have produced for you, had you chosen to generate instead of hand-write it.

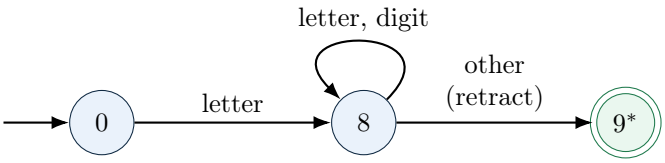
7.2 Transition Diagrams for Individual Token Patterns

Definition: Transition Diagram (DB §3.4 Sense)

A **transition diagram** here is a DFA-like picture built for *one* token category at a time: circles for states, labeled arrows for the character(s) that trigger each transition, and one or more accepting states. It differs from the general DFAs of Lecture ?? in one specific way: some accepting states are marked with a $*$, meaning "this state accepts, but only after the scanner has already read one character *too far* to know that – that extra character must be pushed back before the token is considered complete." This extra bookkeeping step is called **retraction**, covered in §7.3 below.

7.2.1 Worked Example: Identifiers and Keywords

[DB] The simplest and most-reused diagram: one letter, then any number of letters or digits – exactly Lecture ??'s `id` regular definition (§??), just drawn as a picture instead of written as a formula.

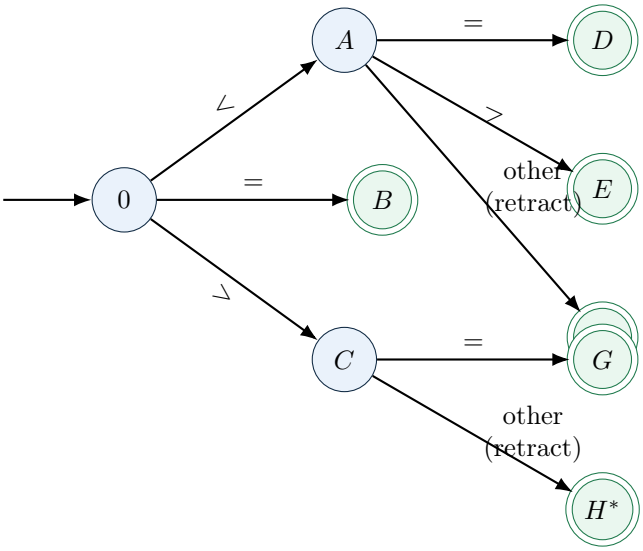


Example: Reading Diagram: Identifier/Keyword

State 8 self-loops on every letter or digit – exactly maximal munch, one character at a time. The moment a character arrives that is *neither* a letter nor a digit (a space, an operator, end of file), that character cannot be part of the identifier, so state 9 accepts – but state 9 is marked $*$, because the scanner already consumed that one extra, non-identifier character to *discover* the identifier was finished. §7.3 explains exactly what happens to that character next. Once the lexeme is known, the reserved-word trick from §?? decides `ID` vs. a specific keyword token via a symbol-table lookup – the diagram itself cannot tell `while` apart from `score`; only the lookup afterward can.

7.2.2 Worked Example: `relop` (DB's Classic Diagram)

[DB] DB's single most-cited transition-diagram example: the six relational operators `<`, `<=`, `<>`, `=`, `>`, `>=`. It is worth tracing by hand once, since it is the clearest possible illustration of *why* retraction is needed at all – some branches consume exactly one extra character to confirm a longer match, others don't.



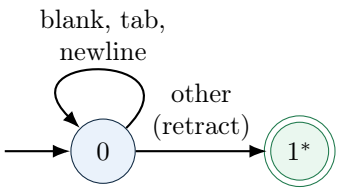
State	Token, Attribute	Retract Needed?
B	`RELOP`, EQ (=)	No – `=` alone is never a prefix of a longer `relop` in this language, so the very next character read is already known to belong to whatever comes *after* this token.
D	`RELOP`, LE (<=)	No – both characters of the lexeme were consumed and both were needed.
E	`RELOP`, NE (<>)	No – same reasoning as D.
F^*	`RELOP`, LT (<)	**Yes** – reaching F means the scanner read `<` *and then* some other character (to rule out `<=` and `<>`) that turned out not to belong to this token at all.
G	`RELOP`, GE (>=)	No.
H^*	`RELOP`, GT (>)	**Yes** – same reasoning as F.

Exam Focus

Notice *exactly* which states need the asterisk, and why: only the two states (F, H) reached by taking the "other" branch out of a two-character lookahead need retraction. States reached by consuming a character that was itself part of a *necessary*, meaningful transition (B, D, E, G) never need it. A very common exam mistake is marking every accepting state with a $*$ out of caution – retraction is needed *only* when a character was read purely to *rule out* a longer match and then turned out not to belong to the token at all.

7.2.3 Worked Example: Whitespace

[DB] One more diagram worth seeing explicitly, because it is the one diagram that produces *no token at all* – exactly §**??**'s "housekeeping" point that whitespace and comments are consumed and discarded, not emitted as tokens.



Example: Why Whitespace Still Needs Retraction

State 0 self-loops through any run of spaces, tabs, or newlines. The instant a non-whitespace character appears, that is exactly the retract situation again: the scanner has read one character belonging to the *next* real token, purely to discover whitespace has ended. State 1* "accepts," but instead of emitting a token, the scanner simply **restarts** matching from state 0 of whichever diagram applies to the retracted character – whitespace is the one pattern in a real lexer specification whose successful match produces zero tokens, not one.

7.3 Retraction and the Buffer, Revisited

[DB] Every $*$ in the diagrams above means the same concrete thing: §**??**'s maximal-munch loop already described this exact situation generically ("keep advancing... roll back to the last remembered accepting position"), and §**??**'s double-buffering scheme is precisely what makes that rollback cheap. Nothing new is being invented here – transition diagrams just make the *single most common form* of that rollback (exactly one character of overshoot) visible and explicit, state by state, instead of leaving it implicit inside a generic simulation loop.

Retraction, Concretely

Reaching a $*$ state means: move the `forward` pointer (§**??**) back by exactly one position before recording where the lexeme ends. Because §**??**'s buffers keep recently-read characters in memory, this is a pointer decrement, not a re-read from disk – exactly why double buffering was worth building in the first place. In hand-written, ad hoc code (§7.4 below), this is often implemented even more directly: use a **non-consuming lookahead** (a `peekChar()` that reads the next character *without* advancing the input pointer) instead of an advance-then-retract pair. The two are behaviorally identical – "peek, then only advance if the character turns out to belong to this token" never overshoots in the first place, so there is nothing left to retract.

Exam Focus

Retraction is also the cleanest illustration of §**??**'s safety condition for reusing a buffer. Every retraction in this lecture steps `forward` back by exactly *one* character, so `lexemeBegin` is never more than one character behind the last-read position – meaning it can never sit further back than the buffer immediately before the one `forward` currently occupies. That is exactly why it is always safe to have already overwritten anything two or more buffers back by the time a retraction happens.

7.4 From Diagrams to Code: The Ad Hoc Approach

[DB] With a transition diagram in hand for every token category, **ad hoc** lexer construction is exactly what the name suggests: translate each diagram directly into ordinary code, by hand, one routine (or one block of a larger routine) per category, dispatched by inspecting the first character of whatever remains in the input.

Definition: Ad Hoc Lexer

An **ad hoc lexer** is a hand-written scanner in which each transition diagram becomes an explicit block of code – typically an `if`/`switch` dispatch on the first character, followed by a small loop or a short chain of character tests that walks that one diagram's states directly, with no shared, generic "simulate a table" loop underneath. The programmer, not a generator tool, is responsible for combining the diagrams (deciding dispatch order, handling the reserved-word trick, wiring

in retraction) correctly.

Example: A Small Ad Hoc Scanner, Sketched

The three diagrams from §7.2 translate almost line for line:

```
Token getNextToken() {
    skipWhitespace();          // Sec. 17-diagram-whitespace, inlined
    c = peekChar();            // non-consuming lookahead (Sec. 17-retraction)

    if (isLetter(c)) {         // Sec. 17-diagram-id
        lexemeBegin = forward;
        do { advance(); c = peekChar(); }
        while (isLetter(c) || isDigit(c));
        lexeme = getLexeme(lexemeBegin, forward);
        if (isKeyword(lexeme))
            return makeToken(keywordTokenFor(lexeme));
        return makeToken(ID, installID(lexeme));
    }

    if (c == '<') {            // Sec. 17-diagram-relop, the '<' branch
        advance();
        if (peekChar() == '=') { advance(); return makeToken(RELOP, LE); }
        if (peekChar() == '>') { advance(); return makeToken(RELOP, NE); }
        return makeToken(RELOP, LT);   // F*/H*: peek already IS the retraction
    }
    if (c == '=') { advance(); return makeToken(RELOP, EQ); }
    if (c == '>') {
        advance();
        if (peekChar() == '=') { advance(); return makeToken(RELOP, GE); }
        return makeToken(RELOP, GT);
    }

    // ... one block per remaining category (NUM, LITERAL, ...) ...

    reportLexicalError(c);
}
```

Notice the F^*/H^* states from §7.2.2 never actually call an explicit "retract" function here at all – using `peekChar()` throughout means the input pointer only ever moves forward past a character once that character is confirmed to belong to the current token, so there is nothing to undo. This is the standard real-world way ad hoc scanners sidestep explicit retraction bookkeeping, exactly as §7.3's coreidea box described.

Exam Focus

Be able to translate a small transition diagram (given to you on an exam) directly into pseudocode of this shape: one `if`/`switch` branch per outgoing edge from the start state, a loop for any self-loop, and a clear point where each accepting state either returns a token or (for whitespace) simply restarts the dispatch. This diagram-to-code translation is exactly what Assignment 1 and Project Phase 1 ask you to do by hand.

7.5 The Table-Driven Approach, Revisited

[DB] Lecture **??** already built this approach in full, so this section is a short recap, not new material: combine *every* token pattern's NFA into one, via the tagged-union construction (§**??**); run subset construction (§**??**) and minimization (§**??**) once, offline; encode the result as a transition table; and drive that table with *one generic loop* – §**??**'s maximal-munch simulation – that has no idea, and does not need to know, how many token categories it is handling. A lexer-generator tool like Flex automates this entire pipeline (§**??**) from a rules file you write, and never once produces or reasons about a hand-drawn diagram like today's.

Exam Focus

Keep straight which lecture built *which* half: Lecture **??** built the **table-driven** half (one combined automaton, one generic simulation loop, machine-built via subset construction). *This* lecture built the **ad hoc** half (many small, separately hand-drawn diagrams, each translated into its own bespoke code). Both are complete, correct ways to end up with a working lexer – they differ in *how* the lexer is constructed, not in what it ultimately does.

7.6 Ad Hoc vs. Table-Driven: A Head-to-Head Comparison

	Ad Hoc (this lecture)	**Table-Driven (Lecture ??)**
How it's built	By hand: draw one diagram per token category, translate each into its own block of code	Automatically: combine all patterns' NFAs, run subset construction and minimization once
Combining many patterns	The programmer's responsibility – dispatch order, reserved-word handling, and retraction must all be gotten right by hand for *every* diagram	Handled uniformly by the tagged-union construction (§**??**) and rule-order tie-breaking (§**??**) – correct by construction
Adding a new token	Write (and correctly integrate) a new block of code, ordered correctly relative to every existing block	Add one line to the rules file and regenerate – the tool re-derives the whole combined automaton
Performance	Can be *very* fast – a human can special-case the hottest patterns (e.g. single-character operators) with no generic overhead at all	Also fast ($O(1)$ per character via table lookup, §**??**), but pays for whatever the combined table's size turns out to be
Correctness guarantees	None automatic – a forgotten retraction, a keyword checked after (not before) the generic `ID` branch, or a missed edge case is a real, easy-to-make bug	Strong – subset construction and minimization are provably correct algorithms; if the rules file is right, the generated scanner is right
Debuggability	Ordinary source code – step through it in any debugger, exactly like any other hand-written function	A generic loop over an opaque table – debugging usually means inspecting *table contents* and current state number, one level more abstract
Best fit	Small, fixed, performance-critical token sets where hand-tuning pays off, or where no generator tool is available/desired	Larger or evolving token sets, and any situation where the specification (the rules file) should be the single source of truth

Extra Depth: The Hybrid Reality

Just as Lecture **??** showed that "compiled vs. interpreted" is rarely a pure either/or choice in practice (most real systems are hybrids), the same is true here. CPython's own tokenizer and V8's (Chrome/Node.js JavaScript engine) lexer are both, in large part, **hand-written** C/C++ code – not output from Flex or a similar tool – precisely because a hand-tuned fast path for the handful of hottest patterns (identifiers, common operators, string-literal boundaries) can outperform a fully generic table-driven loop, while still calling out to smaller, more mechanically-generated pieces (e.g. Unicode category tables) for the parts where hand-tuning would not pay off. GCC's own lexer has, at different points in its history, used both strategies. The lesson: "ad hoc vs. table-driven" is a real engineering choice with real trade-offs (as the table above lays out), not a solved question with one universally correct answer – exactly why this course teaches

both.

Where This Fits: Project Phase 1 and Assignment 1

This lecture is deliberately timed to sit right before you start writing code of your own. **Assignment 1** (regular expressions and finite automata) and **Project Phase 1** (the lexer stage of your semester project, first due around Lecture 7 in the master course outline) both draw directly on today's material: you will draw transition diagrams for your own language's tokens, and then choose – or be told to use – one of the two implementation strategies above to turn those diagrams into a running scanner, before Lecture **??**'s Opt. 1 hands-on session shows the alternative, generator-based route (Flex) side by side with what you just built by hand.

7.7 Self-Check Questions

Practice – Not Graded

1. Draw a transition diagram, in the style of §7.2.2, for a hypothetical `arrow` token family: `-`, `->`, `-` (decrement). Mark every state that needs retraction, and justify each marking.
2. For DB's `relop` diagram (§7.2.2), trace the input `<>=` character by character. Which token is emitted first, and what happens to the remaining `=`?
3. Explain, without saying "because DB says so," why state B (matching bare `=`) in §7.2.2 never needs retraction, while states F^* and H^* always do.
4. Translate the whitespace diagram (§7.2.3) into pseudocode in the same style as §7.4's worked example, and explain why it never calls `makeToken(...)`.
5. A classmate's ad hoc lexer checks the generic identifier pattern *before* checking for specific keywords like `while`. What token does their lexer incorrectly emit for the lexeme `while`, and how does §**??**'s reserved-word trick fix this, concretely, in code?
6. Using §7.6's comparison table, argue *for* using an ad hoc lexer for a small teaching language with 15 fixed token types, and separately argue *against* using ad hoc for a language whose token set is still actively evolving (like an in-development scripting language). What changes between the two scenarios?
7. Explain concretely how using a non-consuming `peekChar()` (§7.3) avoids ever needing an explicit "move the pointer back one" operation, even though every $*$-marked state in this lecture's diagrams conceptually represents exactly that situation.

Key Takeaways

- A **transition diagram** is a small, by-hand DFA for *one* token category; states marked $*$ are exactly the states Lecture **??**'s maximal-munch loop reaches by reading one character past the true end of the token.
- **Retraction** means giving that one overshot character back, so scanning can resume from the right place – cheap in practice because of Lecture **??**'s double-buffered input, and often sidestepped entirely in hand-written code by using a non-consuming `peekChar()` lookahead instead.
- DB's classic `relop` diagram is the cleanest worked illustration of *when* retraction is and isn't needed: only branches that read a character purely to *rule out* a longer match, and

then discover it doesn't belong, need it.

- The **ad hoc** approach translates each diagram directly into hand-written, bespoke code – fast and fully debuggable, but every bit of correctness (dispatch order, keyword handling, retraction) is the programmer's own responsibility.
- The **table-driven** approach (Lecture **??**, in full) combines every pattern into one automaton mechanically and drives it with one generic, provably-correct simulation loop – easier to extend and to trust, at the cost of being one level more abstract to debug.
- Real production lexers (CPython, V8, historically GCC) often **mix both strategies**, hand-tuning the hottest patterns while generating or table-driving the rest – exactly the same "no single universally correct answer" lesson Lecture **??** already drew for compiled vs. interpreted execution.
- This lecture is the direct on-ramp to **Assignment 1** and **Project Phase 1**: drawing your own transition diagrams and choosing an implementation strategy for your own language's tokens.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.